

Descrição Técnica da Arquitetura do Projeto, Pipeline de Dados e Fluxo de Treinamento

ml2_final_project

May 30, 2026

Abstract

Este documento explica o projeto no nível que um novo leitor precisa inicialmente: como o repositório é organizado, como os dados transitam dos corpora brutos para os tensores processados, como o código de treinamento é dividido em módulos reutilizáveis e como as pastas de experimentos são produzidas.

Contents

1 Visão Geral do Repositório

O repositório é dividido em algumas responsabilidades claras: armazenamento de dados brutos, pré-processamento, código reutilizável, saídas de experimentos e documentação.

1.1 Árvore de Diretórios de Alto Nível

```
ml2_final_project/
|-- data/
|   |-- raw/
|   |   |-- libriSpeech-pt/
|   |   |-- tts-portuguese-Corpora/
|   |-- processed/
|   |   |-- libriSpeech-pt/
|   |   |   |-- train/
|   |   |   |   |-- librispeech_mels_metadata.csv
|   |   |   |-- test/
|   |   |   |   |-- librispeech_mels_metadata.csv
|   |   |-- tts_portuguese/
|   |   |   |-- mels_metadata.csv
|   |-- interim/
|   |-- external/
|-- scripts/
|   |-- download_datasets/
|   |-- preprocess/
|   |   |-- preprocess_libriSpeech-pt.py
|   |   |-- preprocess_mels_tts_portuguese.py
|-- src/
|   |-- data/
|   |   |-- first_step_data_loaders/
|   |   |-- datasets.py
|   |-- models/
|   |   |-- AcousticDecoder.py
|   |   |-- GST.py
|   |   |-- TextEncoder.py
|   |   |-- HiFi-GAN.py
|   |-- training/
|   |   |-- train_first_step/
|   |   |   |-- train.py
|   |   |   |-- model_loader.py
|   |   |   |-- losses.py
|   |   |   |-- train_utils.py
|   |   |   |-- text_processing.py
|   |   |   |-- configs.py
|   |   |   |-- run_training.py
|   |   |-- checkpoint_utils.py
|-- experiments/
|   |-- step_1/
|   |   |-- attempt_<timestamp>/
|-- docs/
```

1.2 O Que Cada Pasta Principal Significa

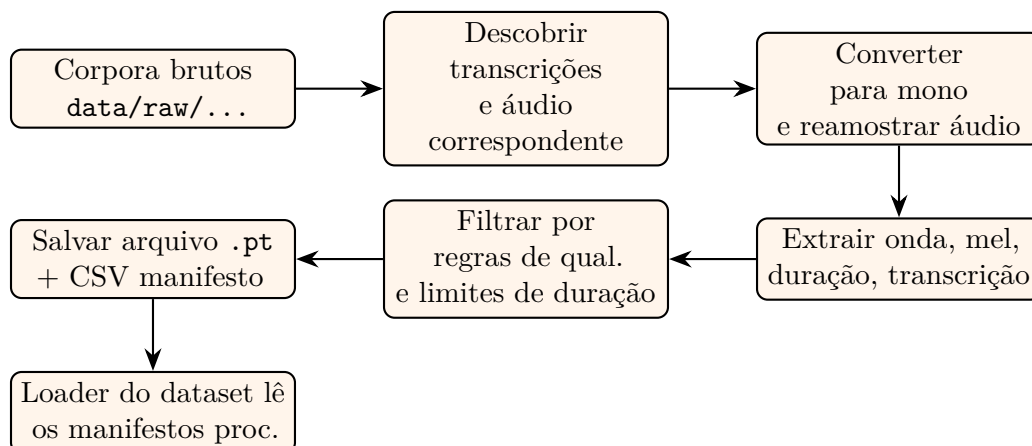
Pasta	Papel no projeto
<code>data/raw</code>	Corpora originais exatamente como baixados. Nada aqui deve depender de uma versão de modelo ou de uma receita de extração de características.
<code>data/processed</code>	Artefatos processados, como arquivos <code>.pt</code> e manifestos CSV de metadados usados pela camada de dataset.
<code>data/interim</code>	Saídas intermediárias temporárias ou arquivos de rascunho que não pertencem ao dataset processado final.
<code>scripts/preprocess</code>	Scripts de pré-processamento reprodutíveis que convertem áudio bruto em amostras de treinamento filtradas, normalizadas e indexadas.
<code>src/data</code>	Wrappers de dataset e auxiliares de batching que transformam manifestos em objetos PyTorch.
<code>src/models</code>	Componentes de redes neurais reutilizáveis. O texto de arquitetura neste documento não depende desses arquivos como sua fonte da verdade.
<code>src/training</code>	Orquestração de treinamento, funções de perda (losses), predefinições de configuração, salvamento de checkpoints e utilitários de experimentos.
<code>experiments</code>	Saídas de experimentos nomeadas ou com timestamp. Cada execução mantém sua configuração, checkpoints, logs e dados do TensorBoard juntos.
<code>docs</code>	Descrições técnicas, relatórios e os códigos-fonte em LaTeX para a documentação do projeto.

2 Fluxo de Dados de Ponta a Ponta

O projeto segue um ciclo de vida simples:

1. Download dos corpora brutos para `data/raw`.
2. Pré-processamento de áudio e transcrições em tensores padronizados.
3. Salvamento de artefatos por locução e manifestos de metadados em `data/processed`.
4. Carregamento dos manifestos processados com as classes de dataset em `src/data`.
5. Construção do experimento de treinamento a partir dos módulos em `src/training`.
6. Salvamento de checkpoints, logs e gráficos em `experiments/step_1`.

2.1 Diagrama do Pipeline de Dados



3 Como os Dados Devem Ser Organizados

O projeto trata os dados brutos e os dados processados como dois produtos diferentes. Dados brutos são a fonte da verdade; dados processados são um cache de treinamento reutilizável.

3.1 Layout dos Dados Brutos

Coloque cada corpus em seu próprio diretório em **data/raw**. Um layout prático é:

```
data/raw/  
|-- libriSpeech-pt/  
|   |-- mls_portuguese_opus/  
|       |-- train/  
|       |-- dev/  
|       |-- test/  
'-- tts-portuguese-Corpora/  
    |-- TTS-Portuguese-Corpus/  
        |-- audio/  
        |-- text/  
        |-- metadata/
```

Os nomes exatos das pastas aninhadas podem diferir dependendo de como o corpus foi baixado, mas a regra é a mesma: o conteúdo bruto permanece agrupado por dataset e não é sobrescrito pelo pré-processamento.

3.2 Layout dos Dados Processados

A área de processados deve espelhar a identidade do dataset e a divisão que foi usada durante o pré-processamento. O projeto atual espera estas saídas principais:

```
data/processed/  
|-- libriSpeech-pt/  
|   |-- train/  
|       |-- <utt_id>.pt  
|       |-- librispeech_mels_metadata.csv  
|   |-- test/
```

```
|         |-- <utt_id>.pt
|         '-- librispeech_mels_metadata.csv
'-- tts_portuguese/
    |-- <utt_id>.pt
    '-- mels_metadata.csv
```

As classes de dataset em `src/data/first_step_data_loaders/datasets.py` usam esses manifestos diretamente. Uma linha de exemplo no CSV seria assim:

```
utt_id,text,duration,mel_path,audio_path
pt_000123,ola mundo,2.83,data/processed/libriSpeech-pt/train/pt_000123.pt,data/raw/
    libriSpeech-pt/.../pt_000123.flac
```

3.3 O Que Entra em um Arquivo .pt

Cada amostra serializada deve manter tudo o que é necessário para treinar ou inspecionar o exemplo mais tarde:

- **waveform**: o tensor da forma de onda mono reamostrada.
- **mel**: o tensor do espectrograma log-mel, com 80 bins mel.
- **sr**: a taxa de amostragem usada pelo pré-processamento, geralmente 22050 Hz.
- **duration**: a duração da locução em segundos.
- **text**: o texto da transcrição após a normalização.
- **utt_id**: o identificador único da locução.
- **audio_path**: o caminho original do áudio relativo à raiz do corpus, quando disponível.

3.4 Parâmetros de Extração de Características

A receita de pré-processamento é fixa para que cada amostra seja compatível com o formato de entrada do modelo:

- Taxa de amostragem: 22050 Hz.
- Bins mel: 80.
- Tamanho da FFT: 1024.
- Tamanho da janela (Window length): 1024.
- Tamanho do salto (Hop length): 256.
- Faixa de frequência mel: 0 Hz a 8000 Hz.

3.5 Regras de Filtragem e Justificativa

Um pipeline prático de pré-processamento deve rejeitar exemplos muito curtos, muito longos ou malformados. Neste projeto, o conjunto de regras pretendido é:

- Duração mínima: 1.0 segundo.
- Duração máxima: 20.0 segundos.
- Apenas forma de onda mono.
- Reamostrado para a taxa de amostragem alvo.
- A transcrição e o áudio devem estar alinhados e não vazios.

Esses limites mantêm o preenchimento (padding) sob controle, reduzem os picos de memória e tornam os formatos dos lotes (batches) mais previsíveis.

4 Pré-processamento Passo a Passo

Os scripts de pré-processamento em `scripts/preprocess` são responsáveis por construir os manifestos usados pelo treinamento.

4.1 Fluxo de Trabalho Típico

1. Ler a raiz do corpus bruto.
2. Encontrar os arquivos de transcrição e enumerar as locuções.
3. Corresponder cada linha de transcrição ao seu arquivo de áudio correspondente.
4. Carregar o áudio com `torchaudio`.
5. Converter o áudio multicanal para mono, se necessário.
6. Reamostrar para 22050 Hz.
7. Computar o espectrograma mel com a configuração STFT fixa.
8. Computar a duração e aplicar o filtro de duração.
9. Salvar os tensores em um arquivo `.pt` por locução.
10. Escrever o manifesto de metadados em formato CSV.
11. Opcionalmente, gerar gráficos ou verificações de sanidade para um subconjunto de amostras.

4.2 Exemplo de Comando de Pré-processamento

```
python scripts/preprocess/preprocess_libriSpeech-pt.py \
  --input-dir data/raw/libriSpeech-pt/mls_portuguese_opus \
  --out-dir data/processed/libriSpeech-pt/train \
  --target-sr 22050 \
  --n-mels 80 \
  --n-fft 1024 \
  --hop-length 256 \
  --win-length 1024 \
  --min-duration 1.0 \
  --max-duration 10.0
```

4.3 Por Que Esta Etapa Importa

Esta etapa é onde o projeto se torna reprodutível. Se o pré-processamento for determinístico e documentado, o mesmo corpus bruto sempre produzirá os mesmos artefatos processados, e as execuções de treinamento posteriores poderão ser comparadas de forma justa.

5 Carregamento de Dataset e Batching

O módulo `src/data/first_step_data_loaders/datasets.py` transforma os manifestos processados em datasets do PyTorch.

5.1 Classes de Dataset

O design atual usa três blocos de construção úteis:

- `LibriSpeechPTDataset`: carrega uma divisão do LibriSpeech-PT, tipicamente `train` ou `test`.
- `TTSPortugueseDataset`: carrega o manifesto processado do TTS Portuguese.
- `CombinedFirstStepDataset`: concatena múltiplos datasets em um único conjunto de treinamento.

5.2 Mistura de Treinamento Padrão

A memória do repositório e os auxiliares de dataset concordam com a composição padrão: LibriSpeech-PT train, LibriSpeech-PT test e TTS Portuguese são combinados para o treinamento da primeira etapa. Isso significa que o código de treinamento pode tratar as três fontes como um único pool de exemplos processados.

5.3 O Que Cada Amostra Retorna

Cada item do dataset retorna um dicionário com as informações centrais necessárias pelo loop de treinamento:

- `mel`
- `waveform`
- `sr`

- duration
- text
- utt_id
- mel_path
- audio_path
- source

5.4 Estratégia de Batching

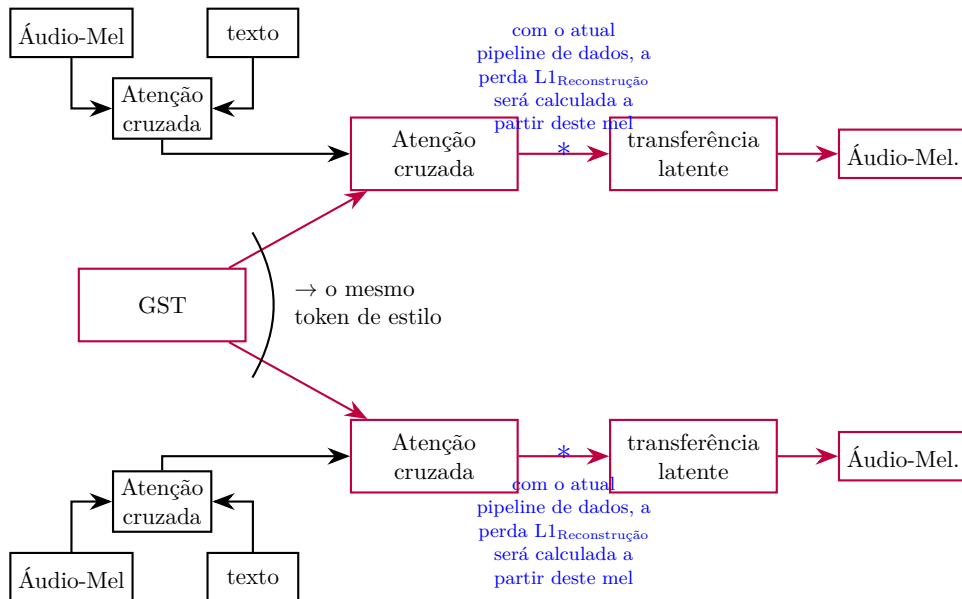
Espectrogramas mel e formas de onda têm comprimentos variáveis, portanto a função *collate* faz o preenchimento (padding) ao longo do eixo do tempo até a amostra mais longa do lote. O lote também mantém metadados como a string de texto, ID da locução, duração e o nome do corpus de origem.

Isso é importante por dois motivos:

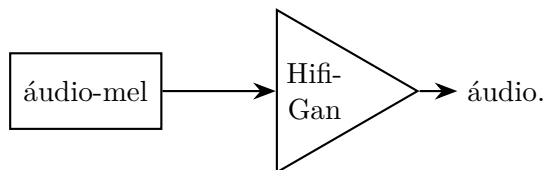
1. Redes neurais esperam tensores com dimensões de lote consistentes.
2. Metadados são necessários para depuração e para análises posteriores do experimento.

6 Arquitetura do Sistema: Estrutura de Treino

A fase de treinamento é projetada como uma arquitetura de caminho duplo que processa características textuais e acústicas. A rede mapeia o áudio em espectrograma mel e o texto correspondente através de mecanismos de atenção cruzada (cross-attention), enquanto simultaneamente extrai características estilísticas via um módulo compartilhado de Token de Estilo Global (GST).



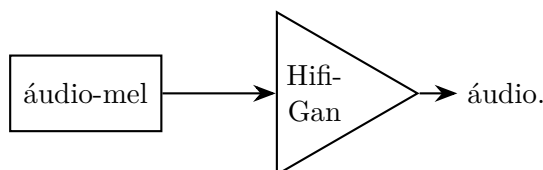
6.1 Diagrama do Pipeline



Durante o treinamento, o módulo central GST processa o áudio-mel para emitir "o mesmo token de estilo", que é injetado em ambos os grandes módulos superiores e inferiores de **Atenção cruzada**. As saídas passam por um módulo de **transferência latente** antes de gerar o **Áudio-Mel** predito. Note que, com o pipeline de dados atual, a perda de Reconstrução L1 será calculada explicitamente a partir do mel extraído na junção marcada (*) antes da transferência latente.

6.2 Diagrama do Vocoder

Para converter os espectrogramas mel de volta em áudio de forma de onda, um vocoder HiFi-GAN é utilizado:



7 Funções de Perda (Loss)

O objetivo de perda combina múltiplos critérios ponderados para governar a qualidade perceptiva, a precisão da reconstrução em ambos os idiomas, a diversidade estilística e a consistência emocional. A fórmula completa é definida estritamente como:

$$\begin{aligned} \text{Loss} = & w_1 \text{Loss}_{\text{perceptual}}(\cdot) \\ & + w_2 L_{\text{Reconstrução}}(\text{Mel-ingles-predito, alvo}) \\ & + w_2 L_{\text{Reconstrução}}(\text{Mel-Português-predito, alvo}) \\ & + w_3 L_{\text{diversidade-estilo}}(\text{todos-os-tokens-de-estilo}) \\ & + w_4 L_{\text{compatibilidade-emocional}}(\dots) \end{aligned}$$

Notas de Implementação:

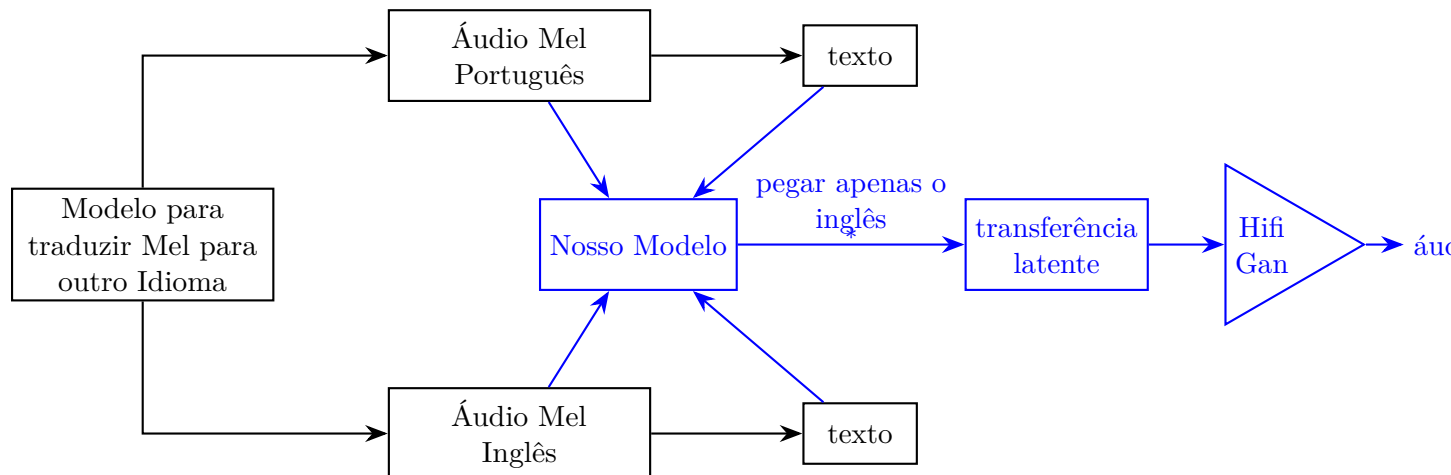
- $\text{Loss}_{\text{perceptual}}$: Avaliada usando a rede de classificação de emoções.
- $L_{\text{diversidade-estilo}}$: Calculada com base na similaridade de cosseno de todos os tokens de estilo.
- $L_{\text{compatibilidade-emocional}}$: Atualmente indefinida, mas planejada para utilizar a rede de classificação para auxiliar na imposição da consistência emocional.
- Note que ambas as perdas de reconstrução (Inglês e Português) compartilham o mesmo escalar de peso w_2 na formulação padrão.

8 Fluxo de Inferência

A etapa de inferência descreve como um idioma traduzido (inglês) é sintetizado usando áudio de um idioma de origem (português). A rede processa o áudio Mel e o texto em dois idiomas, isolando as características em inglês traduzido para a saída.

"Para inferência, pegaremos um áudio em português e emitiremos o em inglês. Nosso modelo é treinado para receber: áudio em português e em inglês. Portanto,"

8.1 Diagrama da Arquitetura de Inferência



** Talvez como um próximo passo possamos construir o nosso próprio*

Como observado, as características do **Áudio Mel Português** e do **Áudio Mel Inglês**, juntamente com seus textos associados, são alimentados em **Nosso Modelo**. O modelo isola a representação em inglês por meio de uma etapa de filtragem ("pegar apenas o inglês") e envia o resultado para o bloco de **transferência latente**. Essa saída latente é então sintetizada na forma de onda final áudio pelo vocoder **Hifi Gan**.

9 Organização do Código de Treinamento em src

O pipeline de treinamento é dividido em pequenos módulos para que cada responsabilidade tenha um lugar principal na base de código.

9.1 Mapa de Módulos

Módulo	O que faz
<code>train.py</code>	Ponto de entrada principal do treinamento. Faz o parser dos argumentos, constrói os datasets, instancia o modelo, executa o treinamento e validação, e escreve as saídas.
<code>model_loader.py</code>	Cria o modelo TTS completo da primeira etapa a partir dos submódulos em <code>src/models</code> .
<code>losses.py</code>	Implementa os termos de objetivo usados pelo treinamento.

<code>train_utils.py</code>	Contém os loops de época, auxiliares de checkpoint, logs do TensorBoard e utilitários de validação.
<code>text_processing.py</code>	Converte strings de transcrição em sequências de tokens.
<code>configs.py</code>	Armazena predefinições de treinamento nomeadas, como teste rápido, balanceado, produção e execuções leves.
<code>run_training.py</code>	Lançador de conveniência que seleciona uma predefinição e repassa seus argumentos para <code>train.py</code> .
<code>checkpoint_utils.py</code>	Comandos utilitários para listar, comparar e inspecionar checkpoints salvos.
<code>test_setup.py</code>	Script de verificação de sanidade para confirmar que o ambiente e o carregamento do modelo funcionam corretamente.

9.2 Por Que Esta Modularização Ajuda

Dividir o código dessa forma torna o projeto mais fácil de manter e mais fácil de revisar.

- Se um bug de pré-processamento aparecer, você só edita os scripts em `scripts/preprocess`.
- Se o plano do modelo mudar, você atualiza a descrição da arquitetura neste documento e o código de treinamento correspondente juntos.
- Se a definição de perda (loss) mudar, você mexe em `losses.py` sem reescrever o loop de treinamento.
- Se você quer uma nova configuração de experimento, você adiciona uma predefinição em `configs.py` em vez de duplicar um comando manualmente.

9.3 Fluxo de Experimento Recomendado

A ordem prática para uma execução é:

1. Verificar o ambiente com o teste de configuração.
2. Pré-processar os corpora brutos em `data/processed`.
3. Inspecionar uma ou duas linhas do manifesto e um ou dois arquivos `.pt`.
4. Lançar uma pequena predefinição como `quick_test`.
5. Verificar o diretório do experimento e os logs do TensorBoard.
6. Aumentar a escala para `balanced` (balanceado) ou `production` (produção) depois que o pipeline estiver estável.

10 Como Rodar os Experimentos

Os experimentos são armazenados em `experiments/step_1`. Cada execução cria um diretório separado, geralmente nomeado com um timestamp ou um nome de experimento customizado.

10.1 Árvore de Diretórios de Experimentos

```
experiments/  
  '-- step_1/  
    '-- attempt_YYYYMMDD_HHMMSS/  
      |-- config.json  
      |-- checkpoints/  
        |-- epoch_0001.pt  
        |-- epoch_0002.pt  
        |-- '-- best.pt  
      |-- tensorboard/  
      '-- logs/
```

10.2 O Que Cada Saída Significa

- `config.json`: os hiperparâmetros exatos usados na execução.
- `checkpoints/`: estados de treinamento salvos que permitem retomar ou avaliar posteriormente.
- `tensorboard/`: arquivos de eventos para curvas de perda, métricas e logs qualitativos.
- `logs/`: logs de treinamento legíveis por humanos ou saída de texto auxiliar.

10.3 Comandos de Exemplo

```
# Teste rapido de fumaca (smoke test)  
python src/training/train_first_step/run_training.py quick_test  
  
# Execucao padrao balanceada  
python src/training/train_first_step/run_training.py balanced  
  
# Treinamento personalizado completo  
python src/training/train_first_step/train.py \  
  --num-epochs 100 \  
  --batch-size 32 \  
  --learning-rate 1e-3 \  
  --weight-reconstruction 1.0 \  
  --weight-diversity 0.5 \  
  --acoustic-decoder-hidden-size 256 \  
  --acoustic-decoder-num-layers 3 \  
  --style-embedding-dim 128 \  
  --num-workers 4 \  
  --experiment-name cross_lingual_training
```

10.4 Monitorando uma Execução

Você pode monitorar o treinamento com o TensorBoard:

```
tensorboard --logdir experiments/step_1/<experiment_name>/tensorboard
```

Curvas úteis incluem perda total, perda de reconstrução, perda de diversidade, métricas de validação e quaisquer exemplos de áudio logados a partir do vocoder.

11 Guia Passo a Passo do Download ao Treinamento

Esta seção é o caminho mais curto de um corpus novo até uma execução de modelo reprodutível.

11.1 1. Baixe os datasets

Coloque os arquivos brutos em `data/raw`. Não faça o pré-processamento antes que a árvore de downloads esteja completa.

11.2 2. Verifique a estrutura de pastas

Certifique-se de que o layout do corpus seja legível pelos scripts de pré-processamento. Se um dataset usar nomes de pastas diferentes, adapte o caminho de entrada antes de executar o script.

11.3 3. Execute o pré-processamento

Use os scripts em `scripts/preprocess` para criar o dataset processado. O resultado deve ser um arquivo `.pt` por locução, mais um CSV de manifesto.

11.4 4. Inspecione algumas saídas

Abra uma linha do manifesto e uma amostra salva para confirmar que o tensor mel, a forma de onda, a duração e a transcrição pareçam corretos.

11.5 5. Confirme que o loader do dataset consegue ler os manifestos

O módulo de dataset deve resolver com sucesso as entradas `mel_path` e retornar dicionários com chaves consistentes.

11.6 6. Comece com um experimento pequeno

Execute `quick_test` primeiro. Ele é projetado para uma rápida verificação de sanidade, não para qualidade final.

11.7 7. Aumente a escala

Depois que a execução pequena funcionar, mude para `balanced` ou para um comando de treinamento personalizado.

11.8 8. Revise os artefatos do experimento

Verifique `config.json`, o melhor checkpoint, as curvas do TensorBoard e quaisquer logs de áudio gerados.

12 Como Modularizar o Projeto de Forma Limpa

Se você deseja estender o projeto sem criar um script monolítico, mantenha cada responsabilidade na mesma camada à qual ela já pertence.

12.1 Adicionando um Novo Dataset

Adicione um novo script de pré-processamento em `scripts/preprocess`, grave os arquivos processados em `data/processed/<novo_dataset>` e crie um wrapper de dataset correspondente em `src/data/first_step_data_loaders`.

12.2 Adicionando um Novo Componente de Modelo

Crie o módulo neural em `src/models`, depois conecte-o na camada de implementação que usa a arquitetura conceitual descrita acima. Se o componente alterar o comportamento da perda, atualize `losses.py` ao mesmo tempo.

12.3 Adicionando uma Nova Variante de Experimento

Coloque os hiperparâmetros em `configs.py`. Isso mantém as invocações de linha de comando curtas e torna os experimentos fáceis de reproduzir.

12.4 Adicionando Melhor Depuração

Gere logs de exemplos, formatos (shapes) e checkpoints a partir de `train_utils.py`. Mantenha a saída de depuração próxima ao loop de treinamento, não espalhada por módulos não relacionados.

13 Exemplo Prático de Todo o Fluxo de Trabalho

Uma execução mínima completa pode ser assim:

1. Faça o download do LibriSpeech-PT e do TTS Portuguese para `data/raw`.
2. Pré-processe ambos os corpora e grave os manifestos em `data/processed`.
3. Inicie `python src/training/train_first_step/run_training.py quick_test`.
4. Confirme se os checkpoints são criados em `experiments/step_1`.
5. Abra o TensorBoard e verifique se as perdas diminuem conforme o esperado.
6. Se tudo estiver estável, execute novamente com a predefinição `balanced`.

14 Resumo

O projeto é organizado em torno de uma regra simples de engenharia: dados brutos ficam em `data/raw`, artefatos processados ficam em `data/processed`, código reutilizável vive em `src` e saídas de experimentos vivem em `experiments`. A arquitetura do modelo neste documento é orientada por diagramas, não orientada por código. Isso mantém a explicação alinhada com o design conceitual que o projeto está tentando comunicar.